

Operational Problem Complexity 研究启动前：第一轮概念确认

先把三个具体问题校准

我先只做了很小范围的事实核对，没有启动你要求的“大范围 literature review”。这样做是为了确保后面我们讨论的对象没有概念错位。

关于 a：OPT-Engine 在利用 LLM 解题时是否调用 solver？——是，但要区分它比较的三种范式。

OPT-Engine 明确区分了 Pure-Text Reasoning (PTR)、Computationally-aided PTR (CompPTR) 和 Solver-Integrated Reasoning (SIR)。其中 **SIR 会让 LLM 生成可执行的 solver code，并调用外部优化求解器**；论文甚至把它的优势解释为“把精确计算负担交给 deterministic solver”。相反，CompPTR 虽然可以调用 Python、SciPy、itertools 等计算工具，却**明确禁止调用外部 solver 做全局优化**。所以，你后面想验证的“formalized model → solver code → solver”链条，OPT-Engine 的 SIR 正好是一个非常直接的参照对象。¹

更重要的是，OPT-Engine 作者自己的结论与你的直觉非常接近：对 SIR 而言，当数值求解交给 solver 后，主要瓶颈变成了**结构是否被正确建模，尤其是 constraints 的 formulation**；论文直接写道，在其测试设置中，如果逻辑结构、约束和目标被正确捕捉，solver 负责得到最优解。²

关于 b：ORAgentBench 的 end-to-end 并不是“把一道 OR 题发给 LLM，让它纯文本直接报答案”。

它所说的 end-to-end 是从现实风格的 operational artifacts 开始：自然语言 brief、多个数据文件、配置文件等，然后 agent 需要理解数据和业务规则、选择建模策略和求解策略、**编写并运行代码**，最后提交一个可被 validator 检查的 decision artifact。其标准运行环境甚至直接提供了 SCIP 等优化工具；agent 有交互执行时间，最后提交的代码还有独立求解时限。³

因此，ORAgentBench 更接近：

NL + data + rules → modeling strategy → solving strategy → implementation/execution → decision artifact,

而不是：

NL problem → LLM mentally solves → answer.

而且 ORAgentBench 明确允许 agent 采用 exact optimization、optimization + repair、decomposition、heuristics、network flow、constraint programming 等不同策略，所以它测试的实际上比“LLM+solver 自动建模”更宽。⁴

关于 c：你对 ORAgentBench 六维 rubric 的疑虑，我初步认为是非常值得追的。

论文列出了六个 construction-time difficulty dimensions：

{solving strategy requirement, formulation structure, constraint coupling, dynamic structure, data scale, problem

作者强调 difficulty 不能简单用 instance size 表示，并将这些维度作为任务构造时的难度描述。 5

但截至我刚才核对的正文和可检索附录，我尚未找到一个足够可复现的 **operational definition**，例如：“constraint coupling=0/1/2/3 分别必须满足什么客观条件”、“formulation structure 与 constraint coupling 如何严格区分”。他们后续确实把每维视为 0 ~ 3 的变量，并用回归分析它们与 pass loss 的关系，但论文自己也明确承认 **constraint coupling 与 formulation structure、dynamic structure 存在相关性**，以至于 coupling 在多变量分析中的系数会被压低。 6

这一点其实非常关键：你担心的“维度定义重叠”并非只是阅读上的错觉，论文的统计分析本身已经暴露了 correlated dimensions 问题。后续正式研究时，我认为我们必须把它列为重点，而不能直接照搬六维 rubric。 4

还有一个术语层面的风险需要提前指出：“operational complexity”在更早的 operations / supply-chain / manufacturing 文献里已经被用于表示生产系统、supplier-customer network 等的运行复杂性，并不等于你现在想定义的“OR problem 对 LLM/agent 的建模与求解难度”。例如 EJOR 早在 2006 年就有 Advances on measuring the operational complexity of supplier-customer systems。 7

所以正式做 literature review 时，我需要同时调查两件事：**有没有与你的概念真正同义的已有定义**，以及**“operational complexity”这个命名是否会产生学术语义冲突**。

我目前对你真正想做的事情的理解

我暂时把你的核心研究问题理解为：

对于一个由自然语言、业务规则和数据描述的 Operations Research problem instance，是否可以定义一个模型无关、可计算、可解释、可实证验证的“operational problem complexity”，用于预测一个 solver-integrated LLM/agent 将该问题可靠地转化为正确并可求解的优化模型、最终得到高质量决策的难度？

并且你希望它最好具有某种类别 computational complexity 的地位：不是简单说“这个 benchmark 的 Hard 类比较难”，而是找到**问题自身的一组结构属性**，这些属性随着问题变化而变化，并且能够预测不同模型、不同 agent architecture 上的 failure probability。

这里马上产生一个非常重要的理论分叉。

假设我们最终发现：

$$P(\text{correct solution} \mid \text{gold formulation}) \approx 1,$$

并且几乎与原问题的 computational hardness 无关，那么 operational complexity 可以主要定义成：

$$C_{\text{op}} \approx C_{\text{semantic/modeling}},$$

即“从现实问题语义到正确数学模型的难度”。

但假设对于 routing、scheduling、stochastic programming、large MIP、nonlinear optimization 等问题，即便 gold mathematical model 已经给出，agent 仍然大量失败于算法选择、求解策略、decomposition、solver 参数、time limit、repair 或 numerical issues，那么就更合理写成某种：

$$C_{\text{op}} = f(C_{\text{modeling}}, C_{\text{solving}}, C_{\text{execution}}),$$

甚至不应该压缩成一个标量，而应首先定义成 complexity profile/vector。

我认为现在最不能提前假定的，恰恰就是这一步。OPT-Engine 的结果支持“对于它覆盖的 LP/MIP 模板，SIR 的主要瓶颈是 formulation”；但 ORAgentBench 则故意扩展到更现实的 setting，并发现即使 submission feasible，仍可能因为 solution quality 不够而失败，同时 agent 还会失败在 solving strategy、repair 和 improvement 上。⁸

所以我需要先确认你到底想定义哪一种 complexity，再决定后面的文献搜索和实验设计。

最需要你确认的定义边界

请你先回答下面这些问题。这里我宁愿现在问得细一些，因为其中任何一个答案不同，最终论文的研究问题都会发生明显变化。

问题	我需要你确认的内容
Q1	你希望 operational complexity 是 problem-intrinsic / model-independent 的属性，还是允许它相对于某个 agent/model 定义？例如“某题对 GPT-5 很难、对未来模型不难”，这算 complexity 改变了吗？
Q2	研究对象究竟是 optimization modeling complexity ，还是完整的 operational problem solving complexity ？也就是说，data understanding、solver selection、debugging、heuristics、replanning 是否属于你想定义的 complexity？
Q3	complexity 的“对象”是哪一个： 自然语言问题 P_{NL} 、 自然语言+数据+business rules 的完整 instance ，还是它背后的 抽象数学问题 instance ？
Q4	你最终强烈希望得到一个类似 $C_{op}(P)$ 的 单一 scalar score ，还是可以接受先得到一个 vector，例如 $(C_{semantic}, C_{constraint}, C_{computational}, C_{dynamic})$ ，之后再研究是否能合理聚合？
Q5	你说“类比 computational complexity”，你的类比重点是哪一个： 严格理论复杂度/复杂度类 ，还是更宽泛的“存在一个可测量、可预测 performance degradation 的 difficulty measure”？这两条路线的理论要求差异极大。

我尤其希望你认真回答 Q1 和 Q4。

因为如果 operational complexity 最终直接定义成：

$$C(P) = -\log P(\text{LLM solves } P),$$

那么它当然很好验证，但它实际上是 **model-dependent empirical difficulty**，很难获得 computational complexity 那种 problem-intrinsic 的意义。

相反，我目前更倾向于一个方向——但暂时不把它当结论：

先从 problem structure 独立定义 complexity $\implies$ 再用跨 LLM/agent 的 error rate 去验证它

也就是说，**accuracy 应该是 criterion validity，而不是 definition 本身。**

我需要确认这是不是也符合你的目标。

你的“gold model 实验” 还需要澄清一层

你提出的实验思路非常重要，但目前“给定 gold formalized model” 仍有至少三种不同实验，而它们回答的问题并不相同。

我们可以把 solver-integrated pipeline 拆成：

$$P_{\text{NL}} \xrightarrow{A} M_{\text{formal}} \xrightarrow{B} C_{\text{solver}} \xrightarrow{C} S \xrightarrow{D} Y.$$

其中：

A: 自然语言/数据 $\rightarrow$ mathematical formulation

B: formulation $\rightarrow$ executable solver implementation

C: solver/algorithm 找 feasible/optimal solution

D: 把 solution 转换成最终要求的 decision artifact

你现在提出的实验实际上可以是：

Condition NL

$$P_{\text{NL}} \rightarrow M \rightarrow \text{code} \rightarrow \text{solution}$$

Condition Gold-Formal

$$M^* \rightarrow \text{code} \rightarrow \text{solution}$$

但我建议后续还考虑一个更强的 control：

Condition Gold-Code

$$\text{code}^* \rightarrow \text{solver} \rightarrow \text{solution}.$$

这样我们才能分别估计：

$$e_A, \quad e_B, \quad e_C,$$

而不是只知道 NL condition 和 Gold-Formal condition 的差值。

这里需要你确认：

Q6: 你说的 gold formalized model, 具体准备给 LLM 什么?

是类似这种数学表达：

$$\min c^\top x$$

subject to

$$Ax \leq b, \quad x_i \in \{0, 1\},$$

以及完整变量/集合/索引定义；

还是已经给出 Pyomo/Gurobi/SCIP-style pseudocode，只要求 LLM 填数据、执行和输出？

这两种实验测的不是同一个东西。

Q7：你希望实验最后验证的是下面哪个更强的命题？

我把它们按强度区分一下：

弱命题： NL $\rightarrow$ formulation 是当前 solver-integrated agents 的主要 error source。

或者：

强命题： 一旦 gold formulation 给定，后续阶段的失败概率在各主要 OR problem classes 中都近似可以忽略，因此 operational complexity 无需考虑 computational/solving complexity。

第二个命题要困难得多，而且我怀疑 routing/scheduling/stochastic/large-scale combinatorial instances 可能会给它制造反例。ORAgentBench 本身就把 modeling strategy 与 solving strategy 明确分成两个 latent decisions，而不是默认后者可以忽略。 ⁹

我需要知道你真正希望实验支持的是哪一个。

关于 constraints，我需要确认你说的“coupling”是哪一种

这是我认为最值得深入发展的部分，但“constraint coupling”至少可以指四种完全不同的东西。

举一个简单模型：

$$x_1 + x_2 \leq 10$$

$$x_2 + x_3 \leq 8$$

$$x_3 + x_4 \leq 6.$$

这里可以从 incidence graph 看出 constraints 通过共享变量发生结构耦合。

但现实业务问题还可能出现：

如果 A 工厂启动，则 B 仓库只有在至少两条运输线路开启时才能供货；如果需求超过阈值，还要优先满足紧急订单。

这里的复杂度不仅在矩阵 sparsity，而在于多个语义规则必须联合推理才能写出正确约束。

因此可能至少存在：

$C_{\text{coupling}}^{\text{algebraic}}$

和

$C_{\text{coupling}}^{\text{semantic}}$

前者可以基于 variable-constraint bipartite graph / hypergraph 定义，例如 degree、density、connected components、treewidth、spectral properties、constraint overlap 等。

后者则涉及从自然语言 rules 到约束之间的 dependency，例如一条 constraint 需要同时组合多少 business concepts、多少 cross-sentence references、多少 conditional relations。

Q8：你所说的 constraint coupling，主要想捕捉哪一种？

- A. formalized model 中的**数学结构耦合**；
- B. natural-language rules 中的**语义耦合**；
- C. 两者都要，而且希望研究 “semantic coupling → mathematical coupling” 的映射关系。

我个人目前认为 **C 最有论文价值**，但要先听你的判断。

还有一个重要的小校准。

你说：

“很多时候 feasibility 相较于 optimality 更难以满足。”

这里我不确定你想表达的准确含义。

因为从数学上，“optimal solution” 必然首先 feasible；而在 ORAgentBench 的统计结果中，作者实际上观察到 **feasibility 普遍高于最终 pass rate**，也就是说不少 agent 能做出 feasible solution，但 objective quality 不够。 ¹⁰

但在 **SIR 自动建模** 的语境下，你可能想表达另一件事：

一旦 formulation 正确，solver 负责 optimality；因此 agent 真正容易出错的是把 constraints 写错，从而造成原业务问题意义上的 infeasibility。

这就与 OPT-Engine 的分析很接近。 ²

Q9：你这里说的 “feasibility 比 optimality 更难”，是不是第二种意思？

即：

不是说标准优化算法寻找 feasible point 本身比寻找 optimum 难，而是说对 LLM 来说，正确表达全部 feasibility conditions 比把一个已经正确 formalize 的模型交给 solver 求 optimum 更难。

这一点请你明确确认，因为后面整套 constraint-complexity hypothesis 会建立在这个区分上。

最后确认文献与论文目标的范围

还有几项会决定我下一轮怎么检索。

Q10：你说 “近两年 OR 领域主要解决的题目”，你的意思是整个现代 OR 领域的主流 problem families，对吗？

也就是不仅调查 LLM-for-OR，而是给你补一套类似：

LP/MILP → scheduling → routing → network optimization → facility location → inventory/revenue management
的知识地图，并针对每类解释：

1. 典型问题是什么；
2. formulation 难在哪里；
3. solver code 是否高度标准化；
4. gold model 给定后是否仍需要明显的算法/求解策略设计；
5. 对你的 operational complexity hypothesis 有什么意义。

还是你只想覆盖目前 LLM+OR benchmark 里实际出现的 problem classes？

Q11: literature review 的年代边界。

我理解你说“近两年”主要适用于 LLM+OR / automated optimization modeling 的最新研究；但 operational complexity 的理论基础应该向前追溯，包括 optimization problem complexity、oracle complexity、model formulation、constraint graph structure、instance hardness、algorithm selection 等经典文献。经典 optimization 文献本身就区分传统 computational/arithmetic complexity 与 oracle complexity，因此如果我们真想做一个类似 complexity 概念，不能只看 2024–2026。 ¹¹

请确认是否接受：

LLM+OR：重点 2024–2026，必要时向前追；
complexity / OR theory：不设时间限制，从 foundational literature 开始追。

Q12: 最终论文倾向是什么？

你更想把这篇工作做成：

AI/ML benchmark & measurement paper
目标类似 NeurIPS / ICML / ICLR / AAAI：定义 complexity、构造 benchmark、跨模型实验、scaling law / diagnostic。

还是：

OR methodology paper
更强调结构定义、数学性质、problem classes、理论和 operational insight，面向 Operations Research / Management Science / INFORMS Journal on Computing 一类社区。

还是一开始就希望设计成两边都能讲得通？

这个选择会明显影响我们是否必须追求一个非常“数学化”的 complexity definition。

你这一轮可以直接按下面格式回答，不必写长解释：

Q1:
Q2:
Q3:

Q4:
Q5:
Q6:
Q7: 弱命题 / 强命题 / 两者逐步验证
Q8: A / B / C
Q9: 是 / 不是, 并说明
Q10: 整个 OR / 仅 LLM+OR
Q11: 接受 / 修改为……
Q12: AI/ML / OR / 两边都要

等这些边界对齐后, 我还会针对你的回答做**第二轮更窄、更技术性的确认**, 尤其会围绕 “operational complexity 到底是 scalar、vector 还是分层定义” “constraint coupling 的候选数学定义” “gold-model experiment 的因果识别逻辑” 继续问; 在这些确认结束之前, 我不会把现在这些直觉提前包装成最终定义。

1 2 8 OPT-Engine: Benchmarking the Limits of LLMs in Optimization Modeling via Complexity Scaling

<https://arxiv.org/html/2601.19924v2>

3 4 5 6 9 10 ORAgentBench: Can LLM Agents Solve Challenging Operations Research Tasks End to End?

<https://arxiv.org/html/2606.19787>

7 European Journal of Operational Research | Vol 171, Issue ...

https://www.sciencedirect.com/journal/european-journal-of-operational-research/vol/171/issue/1?utm_source=chatgpt.com

11 Gradient-Based Empirical Risk Minimization Using Local Polynomial Regression | Stochastic Systems

https://pubsonline.informs.org/doi/10.1287/stsy.2022.0003?utm_source=chatgpt.com